CROC FPGA Aging Monitor

Complete User Manual

Real-Time Monitoring System for FPGA Aging
Experiments Under Ionizing Radiation

Version 2.1
January 2026

Table of Contents

1. Introduction
2. System Requirements
3. Installation
   3.1 Cloning the Repository
   3.2 Windows Setup
   3.3 Linux Setup
   3.4 Generating Vivado Project
4. Running the Application
5. Interface Overview
   5.1 Sensors & Control Tab
   5.2 Aging Analysis Tab
   5.3 Telemetry Tab
   5.4 Log Tab
6. Repository Structure
7. Executing an Experiment
   7.1 Preparation
   7.2 Configuration
   7.3 Running the Experiment
   7.4 Experiment Completion (DONE)
8. System States
9. Degradation Graphs
10. Data Export
11. Troubleshooting

1. Introduction

The CROC FPGA Aging Monitor is a desktop application designed for real-time monitoring of FPGA aging experiments under ionizing radiation. The system automates the detection of timing degradation and manages transitions between bitstreams with different slack margins.

Key capabilities include:

- Real-time monitoring of 128 timing sensors across 4 buses
- Automatic phase detection from bitstream filenames
- Configurable sensor filtering for transition triggers
- Radiation dose tracking and calculation
- Linear regression analysis with future projections
- Comprehensive data logging and CSV export

This manual covers the complete workflow from installation to experiment execution and data analysis.

2. System Requirements

2.1 Hardware Requirements

Component	Specification
FPGA	Xilinx 7-series (Artix-7, Kintex-7) or newer
Control Board	STM32 microcontroller for voltage control
USB Cable	For serial communication with STM32
JTAG Cable	Xilinx Platform Cable or Digilent cable for FPGA programming

2.2 Software Requirements

Software	Version	Purpose
Python	3.12 or newer	Application runtime
Xilinx Vivado	2025.1 or newer	FPGA programming
Git	Any recent version	Repository cloning
USB Driver	Platform-specific	Serial communication

2.3 Supported Operating Systems

- Windows 10 / Windows 11
- Linux (Ubuntu 20.04 LTS or newer recommended)

3. Installation

3.1 Cloning the Repository

First, clone the repository from GitHub:

```
git clone https://github.com/dan-alencar/Aging-V2.git
cd Aging-V2
```

The repository contains all necessary files including pre-generated bitstreams in the out_bitstreams folder.

3.2 Windows Setup

On Windows, the easiest way to run the application is using the provided batch script:

Step 1: Open Command Prompt or PowerShell

Navigate to the repository folder:

```
cd C:\Users\$USER\Aging-V2
```

Step 2: Run the Setup Script

Execute the batch file:

```
run.bat
```

The script will automatically:

1. Check if Python is installed
2. Create a virtual environment (.venv) if it doesn't exist
3. Install all required dependencies from requirements.txt
4. Launch the application

⚠️ **Note:** If you encounter permission issues, run Command Prompt as Administrator.

Step 3: Install USB Driver (if needed)

If the STM32 is not recognized:

1. Open Device Manager
2. Find the unrecognized device under 'Other devices'
3. Right-click and select 'Update driver'
4. Choose 'Search automatically for drivers' or install the STM32 VCP driver manually

3.3 Linux Setup

On Linux, use the shell script for automatic setup:

Step 1: Open Terminal

Navigate to the repository:

```
cd ~/Aging-V2
```

Step 2: Make Script Executable

```
chmod +x run.sh
```

Step 3: Run the Script

```
./run.sh
```

The script performs the same operations as the Windows version.

Step 4: Configure Serial Port Permissions

Add your user to the dialout group for serial port access:

```
sudo usermod -a -G dialout $USER
```

Log out and log back in for changes to take effect.

3.4 Generating Vivado Project (Optional)

If you need to modify the FPGA design or generate new bitstreams:

Step 1: Open Vivado

Launch Xilinx Vivado from the Start menu (Windows) or terminal (Linux):

```
vivado &
```

Step 2: Open the Project

1. In Vivado, select File → Open Project
2. Navigate to: Aging-V2/CROC/
3. Select the .xpr project file
4. Click Open

Step 3: Generate Bitstream

1. In the Flow Navigator, click 'Generate Bitstream'
2. Wait for synthesis, implementation, and bitstream generation
3. The bitstream will be saved in the project's .runs folder
4. Copy the .bit file to out_bitstreams/ with the appropriate naming convention

> 💡 **Tip:** Bitstream naming convention: croc_aging_clkout3phase_mXXX.bit where XXX is the phase value (e.g., m330 for -330°).

4. Running the Application

4.1 Starting the Application

Windows

Double-click run.bat or execute from Command Prompt:

```
run.bat
```

Linux

Execute from terminal:

```
./run.sh
```

Or manually:

```
source .venv/bin/activate
python App/main.py
```

4.2 Initial Configuration

On first launch, configure the following settings:

Serial Port Configuration

1. Go to the Settings sub-tab in Sensors & Control
2. Select the correct COM port (Windows) or /dev/ttyUSB0 (Linux)
3. Set baud rate to 115200
4. Click Connect

Vivado Path Configuration

5. In the same Settings area, locate the Vivado Path field
6. Click Browse and navigate to your Vivado installation
7. Select: C:\Xilinx\Vivado\2023.1\bin\vivado.bat (Windows)
8. Or: /opt/Xilinx/Vivado/2023.1/bin/vivado (Linux)
9. Click Save Settings

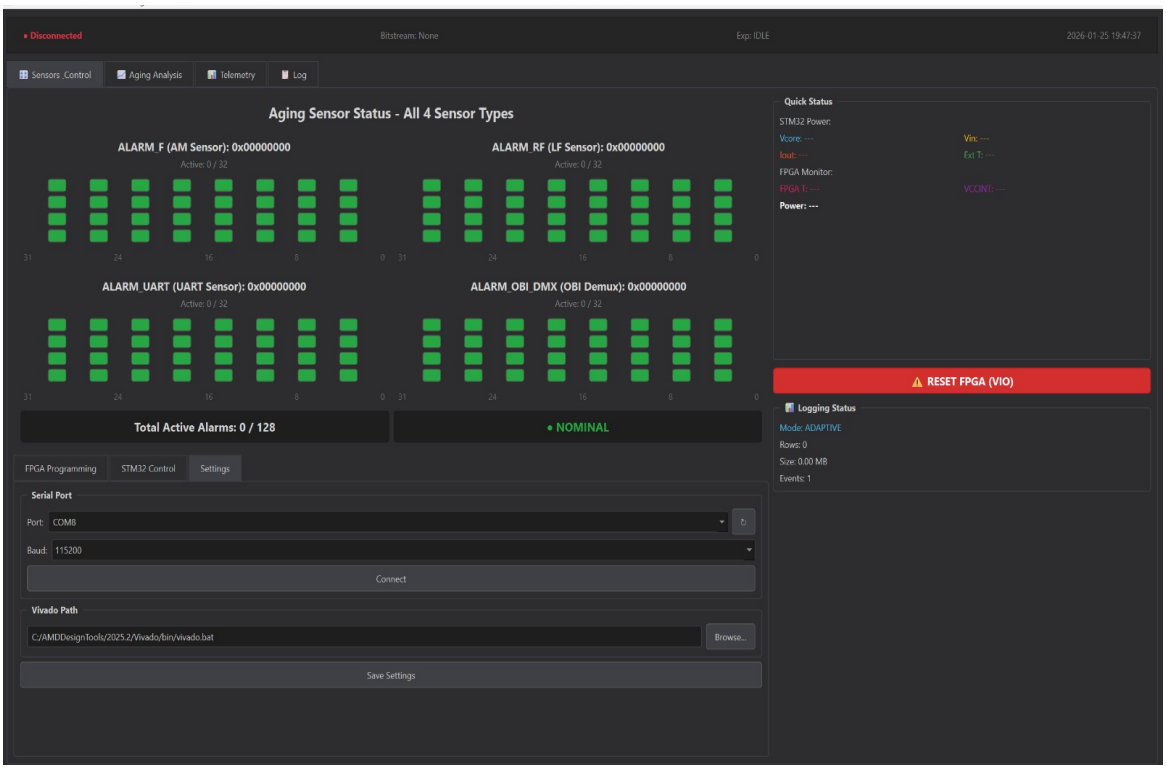

Figure 1: Sensors & Control tab showing connection settings and sensor status

5. Interface Overview

The application interface consists of four main tabs:

5.1 Sensors & Control Tab

This tab displays real-time status of all 128 timing sensors organized in 4 buses:

Sensor Bus	Type	Description
ALARM_F	AM Sensor	Prefetch buffer timing sensors
ALARM_RF	LF Sensor	Register files timing sensors
ALARM_UART	UART Sensor	UART-based timing sensors
ALARM_OBI_DMX	OBI Demux	OBI demultiplexer sensors

Each bus displays 32 bits as colored indicators:

- Green: Normal operation, no timing violation
- Red: Alarm active, timing violation detected

Additional elements on this tab:

- Quick Status: Shows Vcore, Vin, Iout, and temperature readings
- Logging Status: Displays current logging mode, row count, and file size
- FPGA Programming: Controls for manual bitstream programming
- STM32 Control: Direct control of the microcontroller
- Settings: Serial port and Vivado configuration

5.2 Aging Analysis Tab

This is the main tab for configuring and running aging experiments.

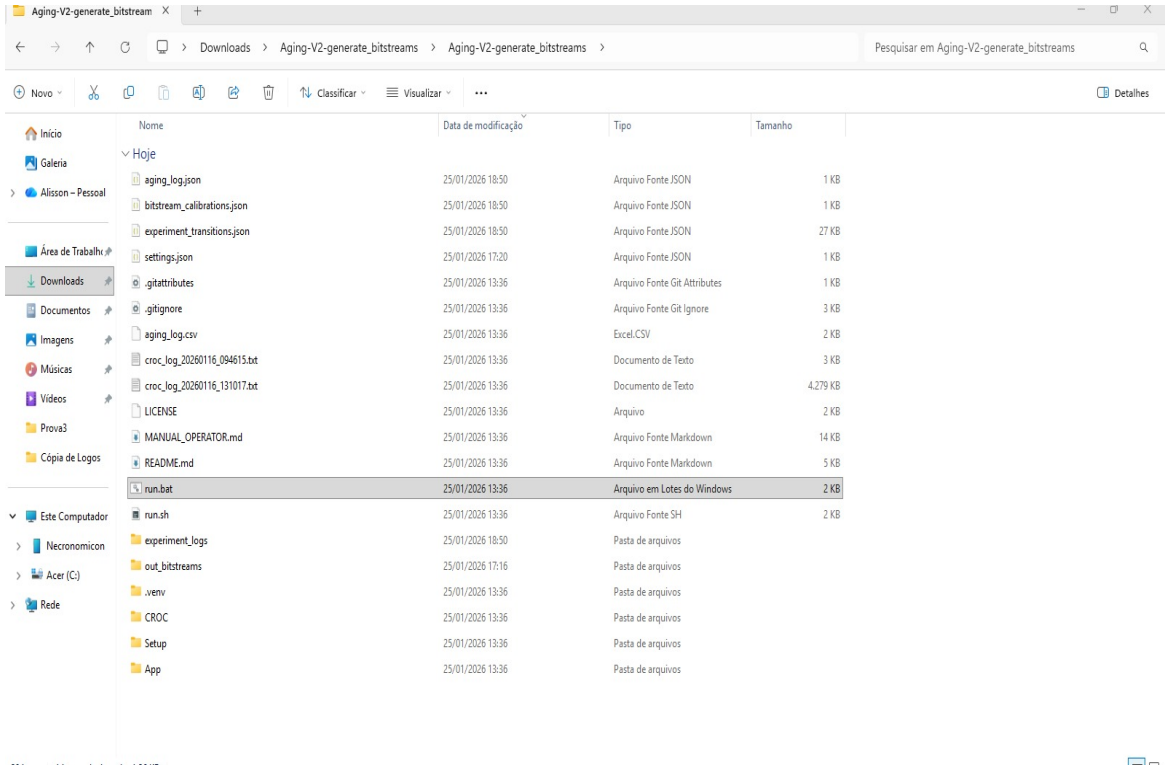

Figure 2: Aging Analysis tab with loaded bitstreams and degradation graphs

Left Panel Components

Bitstream Queue

- Browse: Select the directory containing bitstream files
- Load Bitstreams: Parse and sort bitstreams by phase
- Queue display: Shows all loaded bitstreams with their phases
- Phases are negative (e.g., m330 = -330°, m325 = -325°)
- Bitstreams are sorted from highest to lowest absolute phase

Sensor Trigger Filters

Checkboxes to enable/disable which sensors can trigger transitions:

- F (AM): Prefetch buffer sensors
- RF (LF): Register Files sensors
- UART: UART sensors
- OBI: OBI Demux sensors
- Enable All / Disable All: Quick toggle buttons

> ⚠️ **Note:** Disabled sensors are still monitored and displayed, they just won't trigger automatic bitstream transitions.

Radiation Configuration

- Dose Rate: Radiation source rate in krad/h
- Initial Dose: Pre-accumulated dose before experiment start
- Current Dose: Automatically calculated as Initial + (Rate × Time)

Experiment Control

- Enable Auto-Program on Alarm: Master switch for automatic transitions
- Stabilization: Wait time after alarm detection (milliseconds)
- Cooldown: Wait time after reprogramming (milliseconds)
- State indicator: Shows current experiment state
- Start/Stop/Skip: Experiment control buttons

Right Panel Components

Transition History

Table showing all bitstream transitions with:

- Time: Timestamp of transition
- Hours: Experiment duration at transition
- Dose: Accumulated radiation dose
- From → To: Previous and new bitstream names
- Phase: Phase change in degrees
- Triggers: Which sensors caused the transition
- Alarms: Total number of active alarms

Degradation Graphs (Tabbed)

- Slack vs Dose: Shows slack degradation versus radiation dose
- Slack vs Time: Shows slack degradation versus experiment time

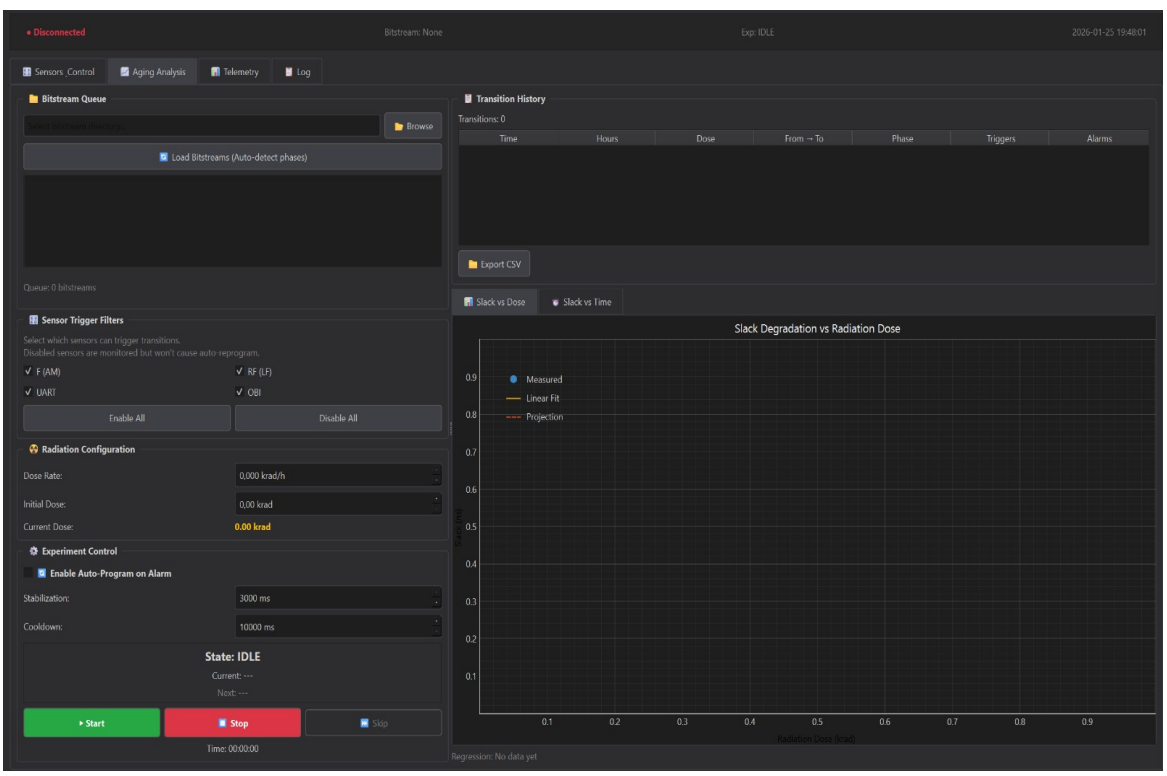

Figure 3: Aging Analysis tab before loading bitstreams

5.3 Telemetry Tab

Displays time-series graphs of environmental and electrical parameters:

- FPGA temperature
- VCCINT voltage
- Sensor alarm counts over time

5.4 Log Tab

Shows detailed application logs for debugging and monitoring:

- Connection events
- Programming operations
- Alarm detections
- State transitions
- Error messages

6. Repository Structure

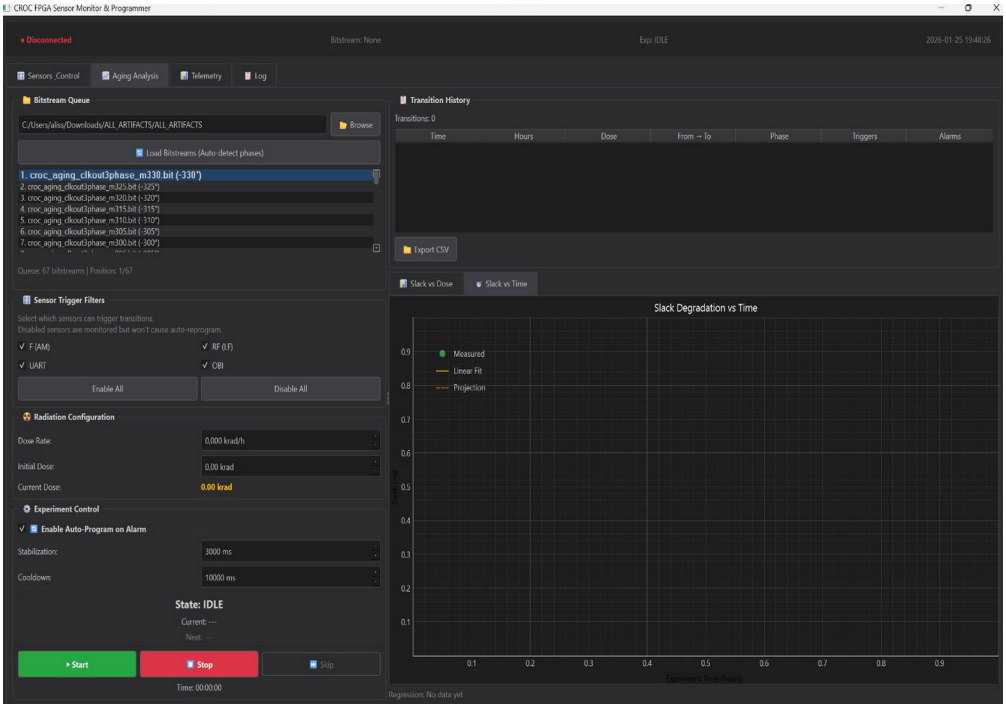

Figure 4: Repository folder structure in Windows Explorer

The repository is organized as follows:

Folder/File	Description
App/	Python source code for the GUI application
out_bitstreams/	Pre-generated bitstreams and probe files (.bit, .ltx)
CROC/	Vivado project with HDL source code
Setup/	Additional setup scripts and utilities
experiment_logs/	Auto-generated log files (created at runtime)
run.bat	Windows launch script with auto-setup
run.sh	Linux launch script with auto-setup
README.md	Project documentation
LICENSE	Project license file

6.1 App/ Folder Contents

File	Purpose
App.py	Application entry point
main_window.py	Main window with all tabs

aging_analysis_widget.py	Aging analysis interface
experiment_controller.py	Experiment state machine
smart_logger.py	Adaptive logging system
sensor_widget.py	Sensor visualization
router.py	Serial communication handler
fpga_manager.py	FPGA programming interface
config.py	Configuration management
requirements.txt	Python dependencies

6.2 out_bitstreams/ Contents

This folder contains the pre-generated bitstreams for aging experiments, for example:

- croc_aging_clkout3phase_m330.bit - Phase -330° (least slack)
- croc_aging_clkout3phase_m325.bit - Phase -325°
- croc_aging_clkout3phase_m320.bit - Phase -320°
- croc_aging_clkout3phase_m315.bit - Phase -315° (most slack)
- Corresponding .ltx probe files for debugging

7. Executing an Experiment

7.1 Preparation

Hardware Setup

10. Connect the MCU board (Serial Decoder) to your computer via USB, and its UART pins to the J9 header in the FPGAging board (refer to the schematic in the repository if needed).
11. Connect the JTAG cable between computer and FPGA
12. Ensure the FPGA board is powered on
13. Place the setup in the radiation environment if applicable

Software Startup

14. Launch the application using run.bat (Windows) or run.sh (Linux)
15. Wait for the interface to load completely
16. Navigate to Sensors & Control tab
17. Select the correct serial port and click Connect
18. Verify the status changes to 'Connected' (green indicator)

7.2 Configuration

Step 1: Load Bitstreams

19. Switch to the Aging Analysis tab
20. Click Browse and navigate to the out_bitstreams folder
21. The path should auto-populate; if not, select manually
22. Bitstreams are automatically loaded and sorted by phase
23. Verify the queue shows all bitstreams in descending phase order

Step 2: Configure Sensor Filters

Decide which sensors should trigger automatic transitions:

- For standard experiments: Keep all sensors enabled
- To isolate specific sensors: Disable others
- If one sensor type is too noisy: Disable it temporarily

Step 3: Configure Radiation Parameters

24. Enter the Dose Rate from your radiation source (e.g., 0.300 krad/h)
25. Enter Initial Dose if the FPGA has prior radiation exposure
26. The Current Dose will be calculated automatically during the experiment

Step 4: Configure Timing Parameters

27. Set Stabilization time (recommended: 3000 ms)
28. Set Cooldown time (recommended: 10000 ms)

> 💡 **Tip:** Longer cooldown times prevent rapid switching when multiple alarms are present.

7.3 Running the Experiment

Starting

29. Check the 'Enable Auto-Program on Alarm' checkbox
30. Click the green 'Start' button
31. The system will program the first bitstream if needed
32. State will change to 'RUNNING' (green)

During Execution

The experiment runs automatically. The system will:

- Continuously monitor all sensor buses
- Detect any active alarms from enabled sensors
- Wait for the stabilization period when alarms are detected
- Program the next bitstream in the queue
- Enter cooldown period after programming
- Resume monitoring

You can observe:

- Real-time sensor status in the Sensors & Control tab
- Transition history in the Aging Analysis tab
- Degradation trends in the graphs
- Detailed logs in the Log tab

Manual Intervention

During the experiment, you can:

- Skip to next bitstream: Click 'Skip' (only available in RUNNING state)
- Modify sensor filters: Check/uncheck sensors at any time
- Adjust timing: Change stabilization or cooldown values
- Stop experiment: Click 'Stop' to pause (data is preserved)

7.4 Experiment Completion (DONE)

The experiment is considered DONE when any of these conditions are met:

Condition 1: Queue Exhausted

- All bitstreams in the queue have been used
- State changes to 'FINISHED' (purple)
- A completion dialog appears with summary statistics

Condition 2: Manual Stop

- User clicks the 'Stop' button
- All data up to that point is saved
- Experiment can be resumed or data can be exported

Condition 3: Error Condition

- Communication failure with hardware
- Programming failure
- Partial data is preserved

Completion Summary

When finished, a dialog displays:

- Total number of transitions
- Total experiment duration
- Final accumulated radiation dose

> ⚠️ **Note:** All data is automatically saved throughout the experiment. You don't need to manually save before stopping.

8. System States

The experiment controller manages the following states:

State	Color	Description
IDLE	Gray	Initial state, experiment not started
PROGRAMMING_INITIAL	Blue	Programming first bitstream at start
RUNNING	Green	Actively monitoring for alarms
ALARM_DETECTED	Yellow	Alarm found, waiting for stabilization
REPROGRAMMING	Blue	Programming next bitstream
COOLDOWN	Gray	Waiting after successful programming
FINISHED	Purple	All bitstreams used, experiment complete

State Transition Diagram

Normal flow:

```
IDLE → PROGRAMMING_INITIAL → RUNNING → ALARM_DETECTED → REPROGRAMMING →
COOLDOWN → RUNNING → ... → FINISHED
```

The cycle (RUNNING → ALARM → REPROGRAM → COOLDOWN → RUNNING) repeats until all bitstreams are exhausted.

Trigger Logic

A transition is triggered when:

33. The experiment is in RUNNING state
34. Auto-Program is enabled
35. Any enabled sensor has an active alarm
36. The cooldown period has completed

> ⚠️ **Note:** The system triggers on ANY active alarm, not just newly detected alarms. This ensures transitions occur even if alarms persist after cooldown.

9. Degradation Graphs

The application provides two graphs to visualize timing degradation:

9.1 Slack vs Radiation Dose

This graph shows how timing slack changes as radiation dose accumulates.

- X-axis: Radiation dose in krad
- Y-axis: Timing slack in nanoseconds
- Blue points: Actual measurements at each transition
- Yellow line: Linear regression fit through data points
- Orange dashed line: Projected future trend

Interpretation:

- Steeper slope indicates faster degradation rate
- Points below the line suggest accelerating degradation
- Points above the line suggest decelerating degradation

9.2 Slack vs Time

This graph shows temporal degradation independent of dose rate.

- X-axis: Experiment time in hours
- Y-axis: Timing slack in nanoseconds
- Green points: Actual measurements
- Yellow line: Linear regression fit
- Orange dashed line: Projected trend

9.3 Regression Information

Below the graphs, regression statistics are displayed:

- Degradation rate: Change in slack per unit (ns/krad or ns/hour)
- R^2 value: Coefficient of determination (0-1, higher is better fit)

Example: 'Slack degradation rate: 0.0234 ns/krad | R^2 = 0.9856'

> 💡 **Tip:** An R^2 value above 0.9 indicates a strong linear correlation between radiation dose and timing degradation.

10. Data Export

10.1 Exporting Transition Data

37. In the Aging Analysis tab, click 'Export CSV'
38. Choose a location and filename
39. Click Save

10.2 CSV Format

The exported CSV contains the following columns:

Column	Description
timestamp	ISO 8601 timestamp of transition
experiment_hours	Hours since experiment start
radiation_dose_krad	Accumulated dose at transition
from_bitstream	Previous bitstream name
from_phase	Previous phase in degrees
to_bitstream	New bitstream name
to_phase	New phase in degrees
total_alarms	Number of active alarms
trigger_f	F sensor alarm bits
trigger_rf	RF sensor alarm bits
trigger_uart	UART sensor alarm bits
trigger_obi	OBI sensor alarm bits
fpga_temp	FPGA temperature at transition
vccint	VCCINT voltage at transition

10.3 Automatic Logging

The system automatically saves:

- experiment_transitions.json: All transition events
- aging_log.json: Detailed experiment log
- Timestamped CSV files in experiment_logs/ folder

11. Troubleshooting

Connection Issues

Cannot connect to serial port

- Verify the USB cable is properly connected
- Check Device Manager (Windows) or dmesg (Linux) for the correct port
- Ensure no other application is using the port
- Try a different USB port
- Reinstall the USB-Serial driver if necessary

Connection drops during experiment

- Check cable connections for loose contacts
- Use a shorter USB cable or USB hub with external power
- Disable USB power saving in Windows Power Options

Programming Issues

FPGA programming fails

- Verify Vivado path is correct in Settings
- Check JTAG cable connection
- Ensure FPGA board is powered
- Try programming manually through Vivado to isolate the issue
- Check that bitstream files are not corrupted

Bitstreams don't appear in queue

- Verify .bit files exist in the selected folder
- Check filename format: must contain phase info (e.g., m330)
- Try clicking 'Load Bitstreams' again

Experiment Issues

System switches bitstreams too quickly

- Increase Cooldown time (e.g., 15000-30000 ms)
- Disable sensors that are triggering excessively
- Check if a specific sensor bus is defective

Alarms detected but no transition occurs

- Verify 'Enable Auto-Program on Alarm' is checked
- Check that the triggering sensor type is enabled in filters
- Confirm state is 'RUNNING' (green)
- Check the Log tab for error messages

Experiment stops unexpectedly

- Check Log tab for error messages
- Verify hardware connections
- Check if all bitstreams have been used (FINISHED state)

Getting Help

If problems persist:

40. Check the Log tab for detailed error messages
41. Export any available data before closing
42. Review the README.md in the repository
43. Open an issue on GitHub with:

- Description of the problem
- Error messages from the Log tab
- Operating system and version
- Vivado version
- Steps to reproduce the issue

© 2026 CROC Project Team